

Integración Continua y Pruebas en E. Urbana

1. Técnicas y herramientas de pruebas según DevOps para E. Urbana

Tipo de prueba	Objetivo en E. Urbana	Técnicas y herramientas	Ejemplo práctico
Prueba unitaria	Validar cada módulo de software de manera aislada (sensores, API, frontend).	Jest (React), (Python scripts para sensores)	Verificar que la API registre correctamente los datos de un sensor.
Análisis de código estático	Detectar errores, vulnerabilidades o malas prácticas antes de ejecutar el código.	Pylint	Revisar que los endpoints de la API cumplan estándares de codificación y no tengan vulnerabilidades de inyección.
Prueba de aceptación	Confirmar que el sistema cumple los requisitos funcionales y expectativas del cliente.	Cucumber, Selenium, Cypress	Simular que un operador revisa en el dashboard las luminarias adaptadas y que se actualizan los datos correctamente.
Prueba de integración	Garantizar que los distintos módulos del sistema funcionan juntos.	Postman	Comprobar que los datos enviados por los sensores se registran en la API y se reflejan en el frontend.
Prueba de rendimiento	Medir tiempos de respuesta y carga del sistema ante muchas peticiones.	JMeter	Simular múltiples solicitudes de datos desde el dashboard para asegurar que no se ralentice.
Prueba de esfuerzo (stress test)	Determinar el límite del sistema bajo cargas extremas.	JMeter	Evaluar qué pasa si cientos de sensores envían datos simultáneamente a la API.

2. Flujos de trabajo (pipeline) de integración continua para E. Urbana

1. **Commit y push:** El desarrollador envía código al repositorio (GitHub).
2. **Build/Compilación automática:** Node.js y React se compilan automáticamente para detectar errores de sintaxis o dependencias.
3. **Análisis de código estático:** ESLint analizan el código y generan reportes de calidad.
4. **Ejecución de pruebas unitarias y de integración:** Jest para frontend, Mocha para backend, pruebas de API con Postman.
5. **Pruebas de aceptación y regresión automatizadas:** Cypress ejecuta tests de interfaz y flujo completo.
6. **Pruebas de rendimiento (opcional según la release):** JMeter simula múltiples usuarios o sensores.
7. **Reporte de resultados:** El pipeline envía notificaciones al equipo.
8. **Deploy automático a staging / producción:** Despliegue seguro si todas las pruebas pasan.

3. Herramientas de automatización CI para E. Urbana

Herramienta	Características principales	Aplicación en E. Urbana
Jenkins	Open source, flexible, permite pipelines complejos, gran cantidad de plugins para Node.js, React y Python	Automatiza compilación, pruebas unitarias e integración, análisis de código estático y despliegue de backend y frontend de E. Urbana
GitLab	Integración nativa con repositorios Gitlab, configuración sencilla mediante YAML, permite pipelines completos	Ejecuta automáticamente tests unitarios, integración y despliegue a entornos de staging/producción al hacer push o pull request en el repositorio de E. Urbana